# Homework 3: The Thread Pool

Spring 2026 CS 3502: Operating Systems

In this assignment you will build a thread pool: a set of worker threads that sit idle until work arrives, process that work concurrently, and shut down cleanly when there is nothing left to do. Thread pools are everywhere in real systems. Web servers use them to handle requests concurrently, and databases use them to run queries in parallel rather than one at a time. Every time your browser loads a page, a thread pool somewhere is doing the work.

Your thread pool will read tasks from standard input, distribute them to a fixed number of worker threads through a shared queue, and print results to standard output. The shared queue is the center of the whole design. It is a shared data structure accessed by multiple threads simultaneously, which means it is a textbook critical section. You will protect it first with a mutex, then add a condition variable so workers can sleep when the queue is empty instead of wasting CPU cycles spinning in a loop.

If you have not forgotten the producer-consumer problem, you already know the theory behind what you are building. The main thread is the producer. The worker threads are the consumers. The queue is the bounded buffer. This assignment turns that theory into code.

Much like HW2, this is not a small project. The levels build on each other and you cannot skip ahead. Level 8 is available as an additional challenge, and is not required for full credit. It is worth a single Level of extra credit beyond the maximum score.

Good luck. Start early. Email me if something seems amiss.

# Setup

This assignment uses the same Linux environment you set up for HW1 and HW2. If you need to configure your environment again, refer to the setup instructions from that assignment.

This assignment supports both x86_64 and ARM (aarch64) architectures. The grader will automatically detect your system and run the appropriate binary. If you are on an ARM machine (such as an M-series Mac running a Linux VM), be aware that I have not tested the ARM build as thoroughly as the x86 build. If you encounter grader issues that seem unrelated to your code, please contact me with the exact error output before assuming your program is wrong.

Download **hw3.zip** from D2L and unzip it in your Linux environment. Inside you will find a starter file called **threadpool.c** and a grader program **hw3_grader**, along with architecture-specific grader binaries. The **hw3_grader** file is a wrapper script that automatically detects whether you are on x86_64 or ARM and runs the correct binary. You only need to interact with **hw3_grader** directly; you can ignore the other grader files.

The starter file contains all of the include headers you will need for every level of this assignment, along with an empty **main** function. Do not remove any of them, even if you are not sure what they are for yet. All of your code goes in **threadpool.c**. You should not need to create any additional files.

To compile your program at any point, run:

```
gcc -o threadpool threadpool.c -pthread
```

The **-pthread** flag is required. It links the POSIX threads library. Without it, any code that uses **pthread_create**, **pthread_mutex_lock**, or related functions will fail to compile. You will forget this flag at least once. When your compile fails with undefined references to pthread functions, this is why.

Your program takes a single command-line argument: the number of worker threads. To run it manually:

```
echo -e "hello\nworld\nfoo" | ./threadpool 4
```

This sends three tasks to a pool of four workers. To check your progress against the autograder:

```
./hw3_grader
```

If the grader returns "Permission denied," run **chmod +x hw3_grader** first. The grader compiles your code, launches it with various inputs and thread counts, checks output correctness and concurrency, and reports which levels pass. When all required levels pass, it outputs a submission hash.

# Level 1: The Task

Before any threading, get the basic task processing working. Your program should read lines from standard input one at a time, process each one, and print a result. This is entirely single-threaded. You are building the foundation that everything else sits on top of.

The task your thread pool will perform is computing a hash of a string. The hash function is called djb2, a simple well-known hash function. It takes a string and returns an unsigned long integer. Add this function to your file above **main**:

```
unsigned long djb2_hash(const char *str) {
    unsigned long hash = 5381;
    int c;
    while ((c = *str++))
        hash = ((hash << 5) + hash) + c;
    return hash;
}
```

You do not need to understand the math. You just need to know that given the same input string, it always returns the same number. That determinism is what makes it useful for grading.

Now write a function called **process_task** that takes a string, sleeps briefly to simulate meaningful work, computes the hash, and prints the result. The sleep is important. In a real system, tasks take time. The artificial delay is how the grader verifies that your pool is actually running tasks concurrently rather than one at a time.

```
void process_task(const char *task) {
    usleep(100000);
    unsigned long hash = djb2_hash(task);
    printf("%lu %s\n", hash, task);
    fflush(stdout);
}
```

The **usleep(100000)** call sleeps for 100 milliseconds. The **fflush** is critical for the same reason it was critical in HW2: when your program's output is piped to the grader, stdout is fully buffered and results will not be sent unless you flush explicitly.

In **main**, start by checking that the user provided a command-line argument. If **argc** is less than 2, print a usage message and return 1. Also check that the thread count is at least 1. If it is not, print a usage message and return 1. Parse the thread count from the first argument using **atoi(argv[1])** and store it in an integer variable. You will not use this number until Level 2, but parse it now so the interface is correct from the start.

Now write a loop that reads lines from stdin using **fgets** into a character array of 1024 bytes. For each line, strip the trailing newline the same way you did in HW2: find it with **strchr** and replace

it with a null terminator. If the resulting string is empty, skip it. Otherwise, call **process_task** with the line. When **fgets** returns NULL, stdin has closed and there are no more tasks. Return 0 from main.

Compile and test. Run **echo -e "hello\nworld" | ./threadpool 1** and confirm you see two lines of output, each containing a number followed by the original string. The order should be hello first, then world, since you are processing sequentially. The whole run should take about 200 milliseconds because each task sleeps for 100.

# Level 2: Spawn Workers

Now create actual threads. This is your first time using the pthreads API, so this level keeps things simple: create the threads, have each one announce itself, and join them all. No shared queue yet.

Every thread needs a function to run. This function must have a specific signature: it takes a single **void*** argument and returns **void***. This is because the pthreads API is generic. It does not know what data you want to pass to your thread, so everything goes through void pointers that you cast to whatever type you need. Write a worker function above **main**:

```
void *worker(void *arg) {
    int id = *(int *)arg;
    printf("Worker %d ready\n", id);
    fflush(stdout);
    return NULL;
}
```

In **main**, after parsing the thread count, declare an array of **pthread_t** to hold the thread handles and an array of **int** to hold the worker IDs. Both arrays should be sized to the thread count. Write a loop that creates each thread. For each index **i**, set the ID array element to **i**, then call:

```
pthread_create(&threads[i], NULL, worker, &thread_ids[i]);
```

Do not pass **&i** directly as the thread argument. The loop variable changes on each iteration, and by the time a thread reads it, the value may have already been overwritten. The separate ID array ensures each thread gets its own stable address to read from.

The first argument is a pointer to the thread handle. The second is thread attributes, which you can leave as NULL for defaults. The third is the function the thread will execute. The fourth is the argument passed to that function. You are passing the address of the ID integer so the worker can read it.

After the creation loop, write a second loop that joins each thread:

```
pthread_join(threads[i], NULL);
```

This blocks until the specified thread finishes. It is the threading equivalent of **waitpid** from HW2. Without it, your main function could return and the process would exit before the workers had a chance to print anything.

Keep your single-threaded task processing loop from Level 1 in place, after the join loop. The workers just print their ready message and return. The actual task processing still happens sequentially in main for now.

Compile and test. Run **echo "hello" | ./threadpool 4** and confirm you see four "Worker N ready" lines followed by the hash result for hello. The worker lines may appear in any order. That is normal and expected. Threads are scheduled by the operating system and there is no guarantee about which one runs first. This nondeterminism is fundamental to concurrent programming and you will be dealing with it for the rest of this assignment.

If your program hangs, you likely forgot to return from the worker function or forgot the join loop. If you see garbled or missing output, make sure you are calling **fflush(stdout)** after every print.

# Level 3: The Shared Queue

Now you are going to build a queue that the main thread pushes tasks onto and the worker threads pull tasks off of. This is what the producer-consumer pattern looks like in code.

First, define the data structures. Each task in the queue is a node in a linked list. The queue itself tracks the head, the tail, and a flag indicating whether the main thread is done reading input. Place these definitions above your worker function so they are visible to all your code:

```
typedef struct task_node {
    char task[1024];
    struct task_node *next;
} task_node_t;

typedef struct {
    task_node_t *head;
    task_node_t *tail;
    int done;
} task_queue_t;
```

Declare a global queue variable and initialize it:

```
task_queue_t queue = { .head = NULL, .tail = NULL, .done = 0 };
```

This is global so that both the main thread and all worker threads can access it. In later levels you will protect this shared access. For now, you are going to leave it unprotected and see what happens.

Now write two functions: **push_task** and **pop_task**. The push function allocates a new node, copies the task string into it, and appends it to the tail of the linked list. If the queue is empty (head is NULL), the new node becomes both the head and the tail. Otherwise, link the current tail to the new node and update the tail pointer:

```
void push_task(task_queue_t *q, const char *task) {
    task_node_t *node = malloc(sizeof(task_node_t));
    strcpy(node->task, task);
    node->next = NULL;
    if (q->tail) {
        q->tail->next = node;
        q->tail = node;
    } else {
        q->head = node;
        q->tail = node;
    }
}
```

The pop function removes a node from the head. If the queue is empty, it returns 0 to indicate there was nothing to pop. Otherwise it detaches the head node, copies the task string into a caller-provided buffer, updates the head pointer, clears the tail if the queue is now empty, frees the node, and returns 1:

```
int pop_task(task_queue_t *q, char *buf) {
    if (q->head == NULL) return 0;
    task_node_t *node = q->head;
    strcpy(buf, node->task);
    q->head = node->next;
    if (q->head == NULL) q->tail = NULL;
    free(node);
    return 1;
}
```

Now restructure your program. Remove the single-threaded processing loop from main. Instead, after reading and stripping each line from stdin, call **push_task** to add it to the queue. After the stdin reading loop finishes, set **queue.done = 1**.

Modify your worker function so that instead of printing a ready message and returning, it runs in a loop. Each iteration, it declares a local character buffer of 1024 bytes and tries to pop a task from the queue. If the pop succeeds, it calls **process_task** on the buffer. If the pop fails (the queue is empty), it checks whether **queue.done** is set. If done is set and the queue is empty, the worker breaks out of the loop and returns. If done is not set, the worker simply loops back and tries again.

Rearrange main so the sequence is: parse arguments, create worker threads, read all input and push tasks to queue, set the done flag, join all worker threads, return. The workers are running the entire time the main thread is reading input and pushing tasks.

Compile and test. Run **echo -e "hello\nworld\nfoo" | ./threadpool 4**.

Here is the important part: **this will probably work most of the time, but it is broken.** You have a shared data structure being accessed by multiple threads with no synchronization. The main thread is pushing while workers are popping. Two workers might try to pop at the same time and both read the same head node. The linked list pointers can get corrupted. You might see duplicate results, missing results, or a segmentation fault. If you run it ten times you might get correct output nine times and a crash on the tenth. Run it with more threads or more tasks and the probability of visible failure goes up.

This is a race condition. We have talked about them plenty. The next level fixes it.

If you happened not to see any crash or corruption in your testing, that does not mean the bug is absent. Race conditions are timing-dependent. The grader may trigger what your terminal did not. Trust that the bug exists and proceed to Level 4.

You may also notice your CPU usage spike during execution. This is expected. Your workers are spinning in a tight loop checking the queue over and over, which wastes enormous amounts of CPU time. Level 5 eliminates this.

# Level 4: The Mutex

A mutex ensures that only one thread can execute a critical section at a time. Your queue operations are the critical section. Every push and every pop must be protected so that only one thread manipulates the linked list at any given moment.

Add a **pthread_mutex_t** field to your queue struct:

```
typedef struct {
    task_node_t *head;
    task_node_t *tail;
    int done;
    pthread_mutex_t mutex;
} task_queue_t;
```

You can no longer fully initialize the queue with a simple aggregate initializer because the mutex requires runtime initialization. In main, before creating any threads, call:

```
pthread_mutex_init(&queue.mutex, NULL);
```

Now wrap every access to the queue's shared state in lock and unlock pairs. The pattern is always the same:

```
pthread_mutex_lock(&q->mutex);
// ... read or write shared data ...
pthread_mutex_unlock(&q->mutex);
```

Your **push_task** and **pop_task** functions should remain bare queue operations with no locking inside them. Instead, every call site must acquire the lock before calling the function and release it afterward. This keeps your queue functions simple and gives you explicit control over the critical section boundaries. In the main thread, lock before pushing and unlock after. In the worker loop, lock before popping and unlock after. Any time you read **queue.done**, you must also hold the lock. In main, when you set **queue.done = 1**, hold the lock.

If you are unsure whether a particular line of code needs the lock, the rule is simple: if it reads or writes any field of the queue struct, it must hold the mutex.

There is one critical performance consideration. Make sure you release the lock before calling **process_task**. Your worker should lock the mutex, pop a task, unlock the mutex, and then process the task. The task processing sleeps for 100 milliseconds. If you hold the mutex during that sleep, every other worker is blocked for the entire duration. You would have a thread pool that processes tasks one at a time, which defeats the purpose entirely. Lock, pop, unlock, then process.

Compile and test with the same inputs as Level 3. The crashes and garbled output should be gone. Results will appear in a consistent (though not necessarily deterministic order) fashion every time you run it. The order of output lines may vary between runs because thread scheduling is nondeterministic, but every input task should appear exactly once in the output.

There is still a problem. When the queue is empty but the main thread has not finished reading input, your workers are spinning in a tight loop: lock, check queue, find it empty, unlock, immediately lock again. This is called busy-waiting. If you run **top** or **htop** in another terminal while your program executes, you will see your CPU usage spike. The workers are burning cycles doing nothing useful. The next level fixes this.

# Level 5: The Condition Variable

A condition variable lets a thread say "wake me up when something changes." Instead of spinning in a loop checking whether the queue has work, a worker can go to sleep and be woken by the main thread when a new task arrives.

Add a **pthread_cond_t** field to your queue struct, next to the mutex. In main, initialize it the same way you initialized the mutex:

```
pthread_cond_init(&queue.cond, NULL);
```

The two operations you need are **pthread_cond_wait** and **pthread_cond_signal**. The wait call takes the condition variable and the mutex. It atomically releases the mutex and puts the calling thread to sleep. When the thread is later woken up, it reacquires the mutex before returning. The signal call wakes up one thread that is currently sleeping on the condition variable.

Modify your worker loop. When a pop fails because the queue is empty and done is not set, instead of immediately looping back to try again, call **pthread_cond_wait**. The worker goes to sleep. It consumes no CPU. When the main thread pushes a task and signals the condition variable, one sleeping worker wakes up, finds itself holding the mutex, and can check the queue again.

There is an important subtlety called a spurious wakeup. A thread can be woken from **pthread_cond_wait** even when no signal was sent. This is a documented behavior of the POSIX API, not a bug. It means you must always re-check your condition after waking. The standard pattern is a while loop rather than an if statement:

```
while (q->head == NULL && !q->done) {
    pthread_cond_wait(&q->cond, &q->mutex);
}
```

If the thread wakes up and the queue is still empty and done is still not set, it goes right back to sleep. The man page for **pthread_cond_wait** (run **man pthread_cond_wait** in your terminal) explains this in detail and is worth reading. If **man pthread_cond_wait** returns nothing, run **sudo apt install manpages-posix-dev**.

Modify your main thread. After each call to **push_task**, signal the condition variable with **pthread_cond_signal** to wake one waiting worker. Do this while still holding the mutex, after the push but before the unlock.

Compile and test. The output should be identical to Level 4 in terms of correctness. The difference is invisible in the output but real in the system: your workers are no longer burning CPU when idle. If you check **top** or **htop** during execution, you will see CPU usage near zero when the queue is empty.

Run the grader at this point. One of the things it tests is concurrency. It sends several tasks to your pool and measures the total wall-clock time. If tasks that should overlap are running one after another (because you are holding the mutex during **process_task**), the time will be too high and the test will fail. If your pool is truly concurrent, multiple tasks process simultaneously and the total time is a fraction of what sequential execution would take.

# Level 6: Clean Shutdown

If you have been testing carefully, you may have noticed that your program sometimes hangs at the end instead of exiting. This happens when a worker thread is blocked inside **pthread_cond_wait** at the moment the main thread finishes pushing all tasks, sets the done flag, and calls **pthread_join**. The worker is asleep. Nobody wakes it up. The join waits forever.

The fix is **pthread_cond_broadcast**. Unlike **pthread_cond_signal**, which wakes one thread, broadcast wakes every thread currently sleeping on the condition variable. After the main thread sets the done flag, it must broadcast so that all sleeping workers wake up, re-check their loop condition, see that done is set, and exit.

The sequence in main after the stdin reading loop should be: lock the mutex, set **queue.done = 1**, call **pthread_cond_broadcast**, unlock the mutex, then join all worker threads.

In your worker function, the while loop condition already checks for done. When a worker wakes from the broadcast, it re-evaluates the loop condition. If done is set and the queue is empty, the while loop exits. At that point the worker should unlock the mutex and return. If done is set but the queue still has tasks in it, the worker should process the remaining tasks before returning. Draining the queue before exiting ensures no submitted work is silently dropped.

Walk through the shutdown sequence in your head for a pool of four workers where two are sleeping on the condition variable and two are actively processing tasks. The broadcast wakes the two sleeping workers. They see done is set, check the queue, and if it is empty they unlock and return. The two active workers finish their current task, loop back, lock the mutex, find done is set and the queue empty, unlock, and return. Main joins all four. The process exits.

Compile and test. Run **echo -e "a\nb\nc" | ./threadpool 8** and confirm the program exits promptly after printing three results. There are more workers than tasks, which means five workers will be sitting idle in **pthread_cond_wait** at shutdown time. If your broadcast is missing or in the wrong place, the program will hang.

Run the grader. It tests shutdown explicitly by sending a small number of tasks to a large pool and enforcing a timeout. If your program does not exit within a few seconds, the test fails.

At this point you also need to clean up your synchronization primitives. After all threads have been joined, call **pthread_mutex_destroy** and **pthread_cond_destroy** on your queue's mutex and condition variable. This is not strictly necessary for correctness on Linux (the resources are released when the process exits), but it is good practice for clean resource management.

# Level 7: Stress Test

No new code is required for this level. The grader runs your thread pool under stress conditions: hundreds of tasks, varying pool sizes from 1 to 32, tasks arriving in bursts, and edge cases like an empty input stream (zero tasks). Your program must produce correct output in all cases. It must never hang or crash, and every input task must appear exactly once in the output.

If you have been careful with your mutex usage and your shutdown logic, this level should pass without changes. If it does not, the most common issues are:

Holding the mutex during **process_task**, which serializes everything and causes a timeout with large inputs. Forgetting to signal after a push, which leaves workers sleeping while tasks pile up. An off-by-one in your shutdown logic that causes one worker to miss the broadcast. A memory leak in your queue nodes that crashes the program under heavy load, which means you forgot to **free** the node after popping it.

The grader output will tell you which specific stress test failed. Use that information to narrow down the problem. Adding temporary debug prints (to stderr, not stdout, so they do not interfere with grader output) can help you trace what each thread is doing. Remember to remove debug prints before final submission.

# Level 8: Dynamic Scaling (Bonus)

This level is optional and worth extra credit.

A fixed-size thread pool has a limitation: if all workers are busy processing slow tasks and the queue fills up, new tasks pile up with nobody to handle them. A real thread pool can respond to this by spawning additional workers when demand is high.

Add a count field to your queue struct that tracks the current number of tasks in the queue. Increment it on push, decrement it on pop. When the main thread pushes a task and the queue depth exceeds a threshold (use twice the base number of workers as a reasonable default), spawn a new worker thread. When a worker pops a task and finds the queue empty, and there are more workers than the original pool size, that worker should exit instead of waiting. This lets the pool shrink back to its baseline when the burst subsides.

You will need to track the total number of active workers with a counter protected by the same mutex. You will also need a dynamically growing array of **pthread_t** handles so you can join every thread you created. The **realloc** function or a simple linked list of thread handles will work for this.

The grader tests this by sending a burst of slow tasks followed by a pause, then checking that additional workers were created during the burst and that the pool shrank afterward. It verifies this by examining the output timing: if the burst tasks overlap more than the original pool size would allow, dynamic scaling is working.

No further guidance is provided for this level. The threading and synchronization primitives you need are the same ones you have already used. The challenge is bookkeeping.

## Submission

Compile your program one final time with **gcc -o threadpool threadpool.c -pthread** and run **./hw3_grader**. The grader tests Levels 1 through 7. The bonus level is tested separately and clearly labeled in the output.

If all required levels pass, the grader prints a submission hash. Create a file called **submission.txt** containing this hash. Submit both **threadpool.c** and **submission.txt** to D2L.

---

## Academic Integrity

You may discuss concepts with classmates, but all code must be typed by you. Copying another student's hash string is detectable and will result in a zero.

---

## Showcase Your Work

A working thread pool is an excellent portfolio piece. If you completed HW2's shell and now this, you have two systems-level C projects that demonstrate real understanding of process management, concurrency, and synchronization. Consider adding this to the same GitHub repository. A README that explains the architecture (producer-consumer queue, mutex protection, condition variable signaling, clean shutdown protocol) shows an interviewer that you can explain why it works, beyond just showing what it does. Most candidates cannot explain what a condition variable does or why a spurious wakeup matters. You can, because you built a system that depends on getting those details right.